TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG
Khoa Công nghệ Thông tin

Bài tập lớn

Mười hai đề tài · Học phần AJP201 · Lập trình Java nâng cao

Đề bài · hình mô tả · rubric chấm

TS. Đỗ Phúc Hảo
Khoa Công nghệ Thông tin, Trường Đại học Kiến trúc Đà Nẵng
Đà Nẵng, 2026

Mục lục

1 Quy định chung **1**
- 1.1 Chọn đề . . . 1
- 1.2 Khung đề, và tại sao nó không phải đề bài . . . 1
- 1.3 Chứng đường: mọi bộ chấm phải tự chứng minh nó còn sống . . . 2
- 1.4 Tiến độ và mốc . . . 3
- 1.5 Nộp bài . . . 3
- 1.6 Về việc dùng AI . . . 3

2 Rubric chấm **4**
- 2.1 Tiêu chí 1: chạy đúng và đủ chức năng (3.0) . . . 4
- 2.2 Tiêu chí 2: vận dụng kỹ thuật Java nâng cao (2.5) . . . 5
- 2.3 Tiêu chí 3: chất lượng thiết kế và mã nguồn (2.0) . . . 5
- 2.4 Tiêu chí 4: kiểm thử và bằng chứng (1.5) . . . 5
- 2.5 Tiêu chí 5: báo cáo và bảo vệ (1.0) . . . 6
- 2.6 Điểm trừ . . . 6

3 Đề 01 — Hệ quản lý mượn trả sách **7**
- 3.1 Bối cảnh . . . 7
- 3.2 Phần lõi: bốn cách một phép mượn có thể thất bại . . . 7
- 3.3 Điểm thiết kế đáng giá nhất của đề . . . 8
- 3.4 Phải làm . . . 8
- 3.5 Chấm thêm . . . 8

4 Đề 02 — Máy chủ chat nhiều người qua TCP **10**
- 4.1 Bối cảnh . . . 10
- 4.2 Phần lõi: TCP không giữ ranh giới bản tin giúp bạn . . . 10
- 4.3 Kiến trúc phải đạt . . . 11
- 4.4 Phải làm . . . 11
- 4.5 Chấm thêm . . . 11

5 Đề 03 — Công cụ phân tích nhật ký máy chủ **12**
- 5.1 Bối cảnh . . . 12
- 5.2 Phần lõi: đường ống xử lý và chỗ nó vỡ . . . 12
- 5.3 Phải làm . . . 13
- 5.4 Chấm thêm . . . 13

6 Đề 04 — Hàng đợi tác vụ và nhóm luồng xử lý **14**
- 6.1 Bối cảnh . . . 14
- 6.2 Kiến trúc và bất biến phải giữ . . . 14
- 6.3 Chỗ bài làm hay treo: dừng hệ . . . 15
- 6.4 Phải làm . . . 15
- 6.5 Chấm thêm . . . 15

7 Đề 05 — Hệ đặt vé xem phim có giữ chỗ **16**
- 7.1 Bối cảnh . . . 16
- 7.2 Phần lõi: đọc rồi ghi không phải một phép nguyên tử . . . 16
- 7.3 Trạng thái một ghế . . . 17
- 7.4 Phải làm . . . 17

7.5 Chấm thêm 17

8 Đề 06 — Trình gộp báo giá từ nhiều nguồn 18
8.1 Bối cảnh 18
8.2 Phần lõi: song song thật và thời hạn thật 18
8.3 Phải làm 19
8.4 Chấm thêm 19

9 Đề 07 — Bộ nhớ đệm LRU và đo tỷ lệ trúng 20
9.1 Bối cảnh 20
9.2 Phần lõi: một cấu trúc không làm được cả hai việc 20
9.3 Thứ tự loại bỏ, chỗ dễ sai nhất 21
9.4 Phải làm 21
9.5 Chấm thêm 21

10 Đề 08 — Mô phỏng chuyển khoản ngân hàng 22
10.1 Bối cảnh 22
10.2 Phần lõi: bốn điều kiện sinh khoá chết đều có mặt 22
10.3 Bất biến: tổng tiền không đổi 23
10.4 Phải làm 23
10.5 Chấm thêm 23

11 Đề 09 — Máy chủ chia sẻ tệp 24
11.1 Bối cảnh 24
11.2 Giao thức phải tự định nghĩa và tự viết tài liệu 24
11.3 Phần lõi: chặn đường dẫn leo ngược 24
11.4 Phải làm 25
11.5 Chấm thêm 25

12 Đề 10 — Công cụ chấm bài tập lập trình tự động 26
12.1 Bối cảnh 26
12.2 Phần lõi: ba việc dễ làm hỏng công cụ chấm 26
12.3 Chuẩn hoá: khác biệt nào vô hại, khác biệt nào không 26
12.4 Phải làm 27
12.5 Chấm thêm 27

13 Đề 11 — Công cụ xếp lịch phòng học 28
13.1 Bối cảnh 28
13.2 Phần lõi: bốn trường hợp chồng lấn 28
13.3 Ba ràng buộc và ba lý do từ chối 29
13.4 Phải làm 29
13.5 Chấm thêm 29

14 Đề 12 — Cờ ca rô hai người qua mạng 30
14.1 Bối cảnh 30
14.2 Phần lõi: chỉ xét quanh ô vừa đánh 30
14.3 Hai trường hợp phải phân biệt được 31
14.4 Kiến trúc: luật tách hẳn khỏi mạng 31
14.5 Phải làm 31
14.6 Chấm thêm 31

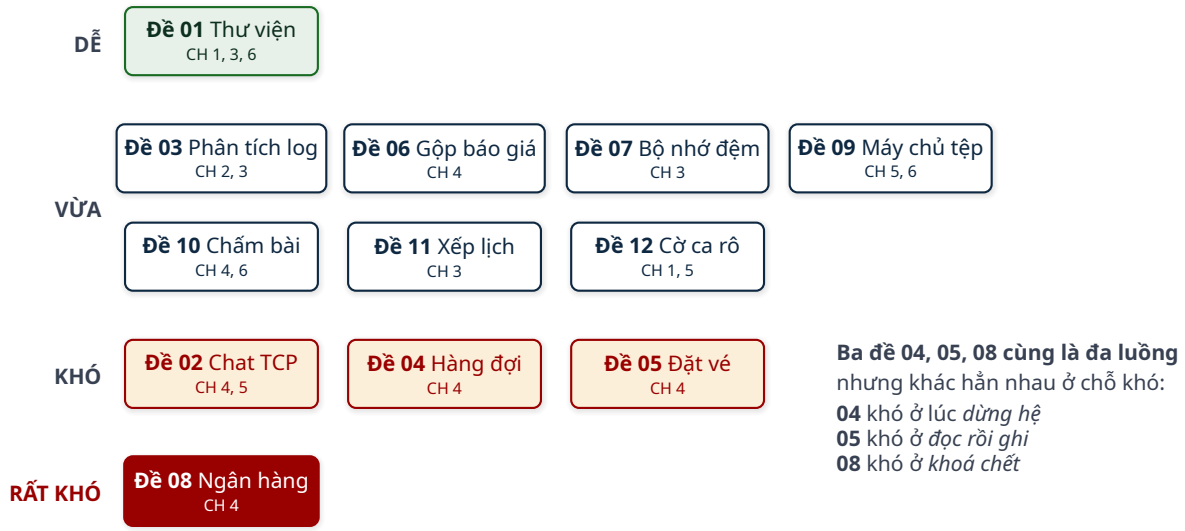
1

Quy định chung

Tập này chứa mười hai đề bài tập lớn của học phần **Lập trình Java nâng cao** (AJP201). Mỗi nhóm chọn một đề. Trong cùng một lớp, không hai nhóm chọn trùng đề. Nhóm gồm hai đến ba người.

1.1 Chọn đề

Mười hai đề không dàn đều về độ khó. Hình 1.1 xếp chúng theo hai trục mà sinh viên thực sự quan tâm khi chọn: đề này nặng về chương nào, và nó khó đến đâu.

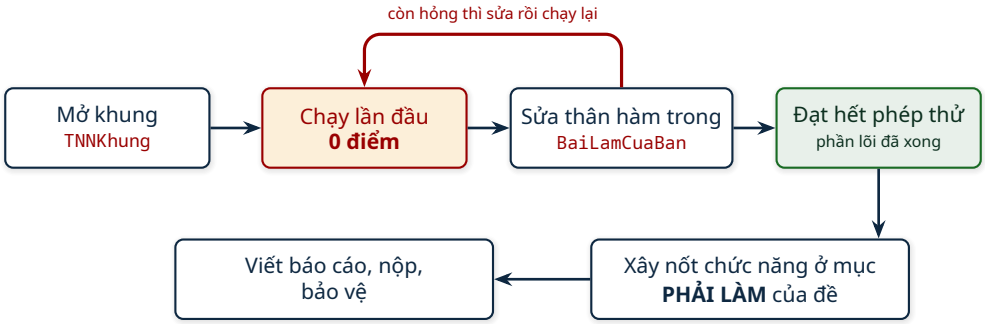


Hình 1.1. Bản đồ mười hai đề theo độ khó và theo chương nặng nhất. Nhãn CH là số chương của giáo trình mà đề dựa vào nhiều nhất.

> **LƯU Ý**
> Đề 08 là đề khó nhất trong mười hai đề. Nhóm nào chọn nó nên biết trước rằng phần lớn thời gian sẽ dành cho việc chứng minh hệ không thể khoá chết, chứ không phải cho việc viết chức năng.

1.2 Khung đề, và tại sao nó không phải đề bài

Mỗi đề đi kèm *đúng một* tệp khung trong dự án Eclipse `AJP201-BaiTapLon`, tên dạng `TNNKhung....java`. Khung chạy được ngay, không cần sửa gì để chạy. Nhưng nó bắt đầu ở **0 điểm**, vì mọi hàm trong lớp `BaiLamCuaBan` đều ném ngoại lệ.



Hình 1.2. Đường đi của một nhóm qua khung đề. Vòng lặp đỏ là chỗ tiêu tốn thời gian nhất, và cũng là chỗ học được nhiều nhất.

Khung chỉ khoá lại **phần lõi** của đề. Phần lõi không phải phần khung sườn, mà là chỗ mà năm năm qua sinh viên đều vấp:

Bảng 1.1. Chỗ sinh viên tưởng là khó, và chỗ thật sự làm hỏng bài.

Đề	Chỗ tưởng là khó	Chỗ thật sự làm hỏng bài
02	Mở `Socket`	Hai bản tin dính vào nhau trên dòng byte
04	Tạo nhóm luồng	Dừng hệ trong khi luồng còn đang chờ
05	Giao diện chọn ghế	Đọc rồi ghi không phải một phép nguyên tử
08	Tính lãi suất	Khoá chết khi hai lệnh ngược chiều gặp nhau
11	Thuật toán xếp lịch	Phép kiểm chồng lấn thiếu một trường hợp
12	Vẽ bàn cờ	Đếm năm quân sai ở đầu mút

GHI NHỚ

Đạt hết phép thử của khung là *điều kiện cần*, không phải điều kiện đủ. Bài nào đạt hết phép thử nhưng thiếu chức năng ở mục **PHẢI LÀM** thì theo rubric chỉ được mức 3 trên 5 ở tiêu chí thứ nhất.

1.3 Chứng đường: mọi bộ chấm phải tự chứng minh nó còn sống

Cuối mỗi tệp khung có mục `CHUNG DUONG`. Nó làm một việc rất đơn giản: đưa cho chính bộ chấm một lời giải *cố ý sai* và đòi hỏi bộ chấm phải báo hỏng.

Lý do có mục này là một bài học lớn hơn cả đề bài: một bộ kiểm thử bị hỏng luôn báo **ĐẠT** rất thuyết phục. Nếu bộ phép thử của bạn chưa bao giờ thất bại, bạn không biết nó đang kiểm cái gì, hay có kiểm gì không.

Vì vậy tiêu chí thứ tư của rubric đòi hỏi chính điều này ở mức 5: bạn phải cố ý làm hỏng một chỗ trong bài mình và cho thấy phép thử của bạn bắt được.

1.4 Tiến độ và mốc



Hình 1.3. Năm mốc của bài tập lớn. Mốc tuần 10 là mốc quyết định.

Mốc tuần 10 quan trọng vì nó đo đúng phần rủi ro. Nhóm nào chưa qua được khung ở tuần 10 thì gần như chắc chắn không kịp tuần 12, và nên đổi sang đề dễ hơn ngay lúc đó chứ không phải tuần 11.

1.5 Nộp bài

Bảng 1.2. Danh mục phải nộp.

Mục	Yêu cầu
Mã nguồn	Một dự án Eclipse xuất ra tệp `zip`, hoặc một kho `git`
Báo cáo	PDF, tối đa 12 trang, theo đúng các mục của rubric
Nhật ký nhóm	Ai làm gì, tuần nào. Một trang
Khai báo AI	Công cụ nào, cho phần nào, nhắc gì
Trình bày	10 phút mỗi nhóm, 5 phút hỏi

Lệnh chạy phải chạy được đúng như ghi trong báo cáo. Nếu báo cáo ghi một lệnh mà lệnh đó không chạy thì trừ điểm theo rubric.

1.6 Về việc dùng AI

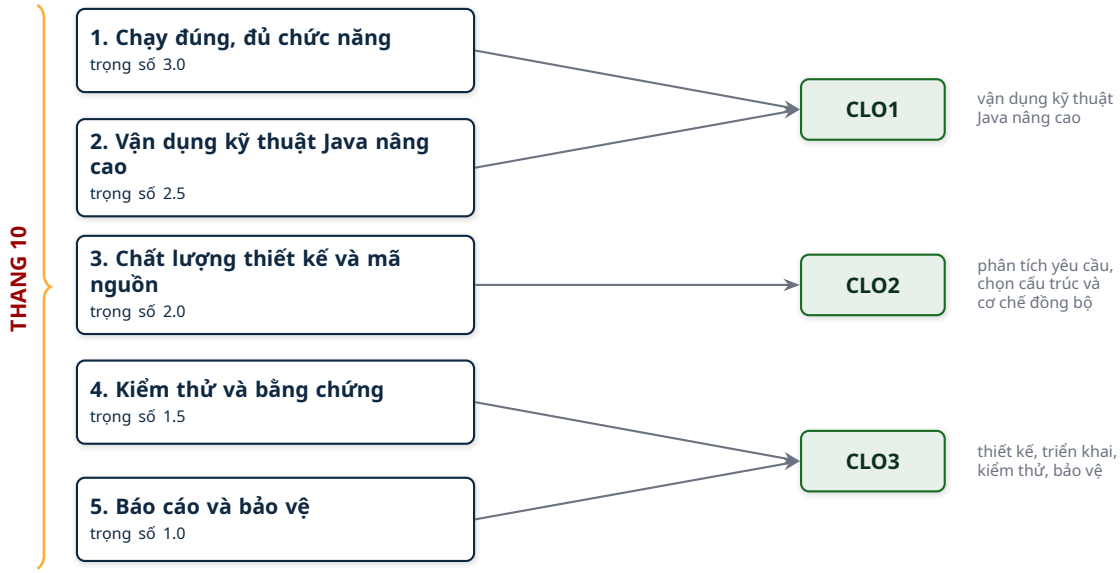
Được dùng. Phải khai báo trong báo cáo: dùng công cụ nào, cho phần nào, nhắc gì. Khi bảo vệ, mỗi thành viên phải giải thích được bất kỳ dòng nào trong bài nộp của nhóm, kể cả dòng do AI viết ra.

Quy tắc này không nhằm chống AI. Nó nhằm bảo đảm một điều đơn giản: bạn chịu trách nhiệm cho thứ bạn nộp.

2

Rubric chấm

Thang 10, gồm năm tiêu chí. Mỗi tiêu chí chấm theo năm mức, mức 1 là thấp nhất. Điểm cuối là tổng điểm từng tiêu chí theo trọng số.



Hình 2.1. Năm tiêu chí và chuẩn đầu ra học phần mà mỗi tiêu chí phục vụ.

2.1 Tiêu chí 1: chạy đúng và đủ chức năng (3.0)

Mức	Mô tả
1	Không dịch được, hoặc chỉ chạy được một phần nhỏ
2	Chạy được luồng chính nhưng hỏng khi gặp dữ liệu bất thường
3	Đủ chức năng bắt buộc, khung T đạt hết phép thử
4	Như mức 3, cộng xử lý đúng các trường hợp biên đã nêu trong đề
5	Như mức 4, cộng ít nhất một phần mở rộng tự chọn, chạy ổn định nhiều lần

Ghi chú chấm. Khung T là điều kiện cần, không phải điều kiện đủ. Đạt hết phép thử của khung mà thiếu chức năng ở mục **PHẢI LÀM** thì cao nhất là mức 3.

2.2 Tiêu chí 2: vận dụng kỹ thuật Java nâng cao (2.5)

Mức	Mô tả
1	Viết như Java cơ bản, không dùng kỹ thuật nào của học phần
2	Có dùng nhưng dùng sai chỗ, ví dụ Stream để làm việc của một vòng lặp đơn
3	Dùng đúng các kỹ thuật mà đề yêu cầu
4	Như mức 3, cộng giải thích được *vì sao* chọn cấu trúc và công cụ đó
5	Như mức 4, cộng có bảng so sánh với một lựa chọn khác, kèm số đo

Ghi chú chấm. Câu hỏi bản lề khi bảo vệ là *nếu đổi sang X thì hỏng ở đâu*. Trả lời được thì từ mức 4 trở lên, không trả lời được thì tối đa mức 3.

2.3 Tiêu chí 3: chất lượng thiết kế và mã nguồn (2.0)

Mức	Mô tả
1	Toàn bộ trong một lớp, tên biến một chữ cái
2	Có tách lớp nhưng phần mạng hoặc giao diện trộn với phần nghiệp vụ
3	Tách lớp hợp lý, phần nghiệp vụ kiểm thử được mà không cần mở mạng
4	Như mức 3, cộng một cách diễn đạt thất bại *duy nhất* trong cả hệ
5	Như mức 4, cộng xử lý tài nguyên đúng cách, không rò rỉ, không biến toàn cục

Ghi chú chấm. Với đề 02, 09 và 12: nếu phần luật hay giao thức không thể kiểm thử được nếu không mở `Socket` thì tối đa là mức 2.

2.4 Tiêu chí 4: kiểm thử và bằng chứng (1.5)

Mức	Mô tả
1	Không có phép thử nào
2	Chỉ có ảnh chụp màn hình chạy tay
3	Có phép thử tự động cho phần lõi, chạy lại được
4	Như mức 3, cộng phép thử cho trường hợp biên và trường hợp hỏng
5	Như mức 4, cộng **chứng đường**: chứng minh được bộ phép thử bắt được lỗi

⚠ LƯU Ý

Mức 5 đòi hỏi sinh viên cố ý làm hỏng một chỗ và cho thấy phép thử báo hỏng. Một bộ phép thử không bao giờ thất bại là một bộ phép thử chưa được kiểm.
Với ba đề đa luồng 04, 05 và 08: phép thử chạy một lần *không* được tính là bằng chứng. Phải chạy lặp ít nhất năm lần và báo cáo cả năm kết quả.

2.5 Tiêu chí 5: báo cáo và bảo vệ (1.0)

Mức	Mô tả
1	Không nộp báo cáo
2	Báo cáo chép lại đề bài và dán ảnh màn hình
3	Báo cáo nêu được thiết kế, cách chạy, và kết quả kiểm thử
4	Như mức 3, cộng mục *những gì đã thử mà không được và vì sao*
5	Như mức 4, cộng bảo vệ được trước câu hỏi phản biện về từng dòng mã nguồn

2.6 Điểm trừ

Lỗi	Trừ
Nộp muộn	1 điểm mỗi ngày, quá 3 ngày không nhận
Mã nguồn không dịch được bằng lệnh đã ghi trong báo cáo	1.5
Sao chép bài nhóm khác	0 điểm cả hai nhóm
Dùng AI mà không khai báo	0 điểm tiêu chí 5
Dùng AI có khai báo nhưng không giải thích được mã nguồn khi bảo vệ	0 điểm tiêu chí 2 và 5

3

Đề 01 — Hệ quản lý mượn trả sách

◎ MỤC TIÊU CHƯƠNG

Độ khó: dễ · **Chương nặng nhất:** 1, 3, 6 · **Khung:** `src/de01_thuvien/T01KhungThuVien.java`

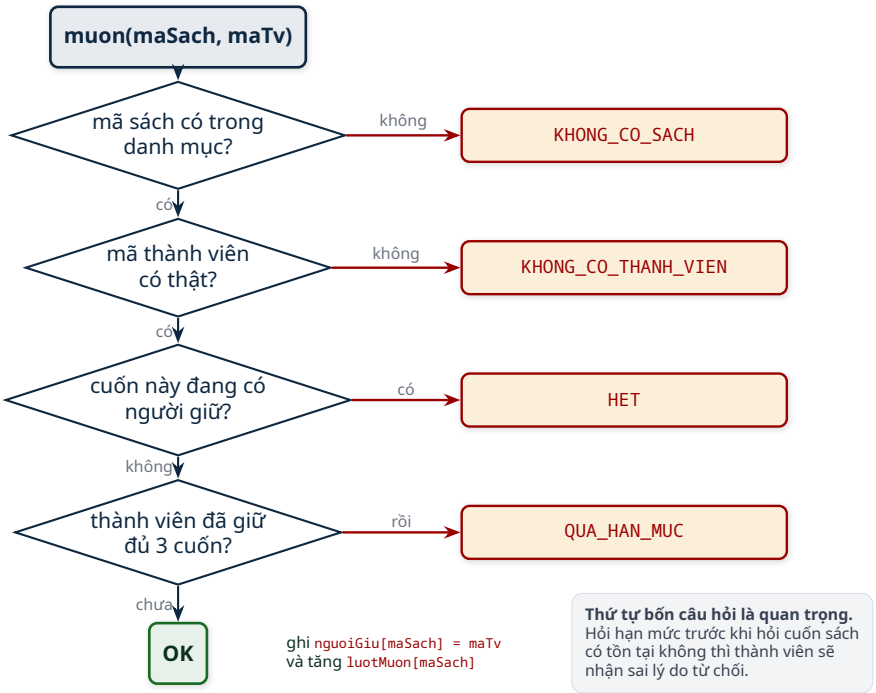
Đề này nhẹ về kỹ thuật và nặng về thiết kế. Điểm số phụ thuộc nhiều vào việc bạn có giữ được *một* cách diễn đạt thất bại duy nhất trong toàn bộ hệ hay không.

3.1 Bối cảnh

Một thư viện khoa có khoảng vài nghìn đầu sách và vài trăm thành viên. Thủ thư quản lý danh mục, thành viên mượn và trả, và cuối kỳ cần báo cáo sách nào được mượn nhiều nhất.

3.2 Phần lõi: bốn cách một phép mượn có thể thất bại

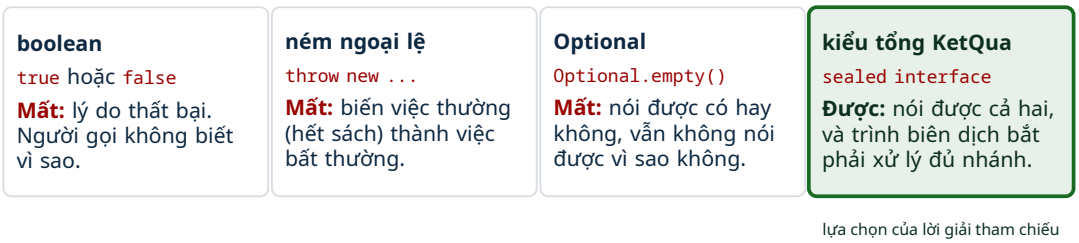
Phép mượn nhìn thì đơn giản, nhưng nó có *năm* kết cục chứ không phải hai. Hình 3.1 là toàn bộ cây quyết định mà khung đã khoá lại bằng phép thử.



Hình 3.1. Cây quyết định của phép mượn. Bốn nhánh đỏ là bốn lý do từ chối, mỗi lý do có một phép thử riêng trong khung.

3.3 Điểm thiết kế đáng giá nhất của đề

Khi một phép mượn thất bại, bạn nói cho người gọi biết bằng cách nào? Có bốn lựa chọn, và cả bốn đều *chạy được*. Chúng khác nhau ở chỗ mất gì.



Hình 3.2. Bốn cách diễn đạt thất bại. Chọn cách nào cũng được, nhưng phải giữ nhất quán trong cả hệ.

GHI NHỚ

Bài nộp trọn điểm là bài mà *mọi* phép nghiệp vụ đều trả về cùng một kiểu kết quả. Bài dùng `boolean` cho `muon` và ném ngoại lệ cho `tra` thì mất điểm tiêu chí 3, dù cả hai đều chạy đúng.

3.4 Phải làm

PHẢI LÀM

- **Danh mục sách:** thêm, sửa, gỡ, tìm theo tên hoặc tác giả, không phân biệt hoa thường.
- **Thành viên:** thêm, xem số sách đang giữ, xem lịch sử mượn.
- **Mượn và trả**, với hạn mức 3 cuốn mỗi thành viên.
- **Báo cáo:** sách mượn nhiều nhất, số sách theo tác giả, sách đang bị giữ. Phần báo cáo bắt buộc dùng Stream, không dùng vòng lặp thủ công.
- **Lưu ra tệp và nạp lại** khi khởi động. Tệp phải chịu được tên sách có dấu phẩy.

TRƯỜNG HỢP BIÊN

Mã sách không tồn tại · mã thành viên không tồn tại · mượn cuốn đang có người giữ · trả cuốn mình không mượn · mượn khi đã đủ hạn mức · tên sách chứa dấu phẩy hoặc dấu nháy kép khi lưu ra CSV.

3.5 Chấm thêm

Báo cáo xếp hạng phải *ổn định*: chạy hai lần trên cùng dữ liệu phải cho cùng kết quả, kể cả thứ tự các mục bằng điểm nhau. Đây là chỗ mà `groupingBy` một tham số làm hỏng bài một cách âm thầm, vì nó trả về `HashMap`.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá bốn nhánh từ chối của `muon`, phép `tra` với cuốn mình không giữ, phép đếm số sách đang giữ, và xếp hạng sách mượn nhiều nhất có phân định khi bằng lượt. Chín phép thử.

Đề này có lời giải tham chiếu hoàn chỉnh tại `src/loigiai/D01LoiGiaiThuVien.java`, gồm 18 phép thử. Giảng viên chỉ phát nó sau khi đã chốt đề, để dùng làm mốc chấm và để nhóm đối chiếu sau khi đã tự làm.

Mở rộng tự chọn. Hạn trả và tính phí trễ hạn. Đặt trước một cuốn đang bị mượn, khi cuốn đó được trả thì người đặt trước được ưu tiên.

4

Đề 02 — Máy chủ chat nhiều người qua TCP

◎ MỤC TIÊU CHƯƠNG

Độ khó: khó · **Chương nặng nhất:** 4, 5 · **Khung:** `src/de02_chatmang/T02KhungChatMang.java`

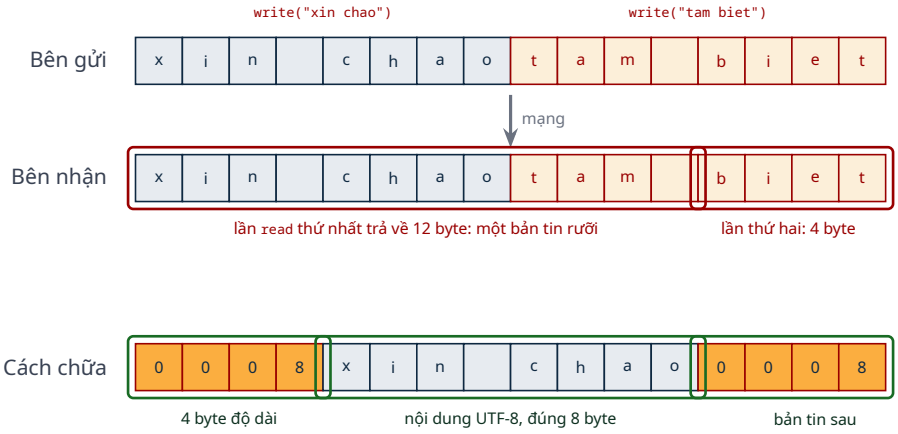
Phần làm sinh viên mất nhiều thời gian nhất không phải `Socket`, mà là ranh giới bản tin trên dòng byte.

4.1 Bối cảnh

Một phòng chat nội bộ cho lớp học. Người dùng chạy chương trình khách ở dòng lệnh, đặt biệt danh, vào phòng, và gửi tin nhắn. Máy chủ phát lại tin nhắn cho mọi người trong cùng phòng.

4.2 Phần lõi: TCP không giữ ranh giới bản tin giúp bạn

Đây là hiểu lầm phổ biến nhất của cả học phần. Khi bạn gọi `write` hai lần, bên nhận *không* bảo đảm sẽ đọc được thành hai lần. TCP là một *dòng byte*, không phải một dòng bản tin.

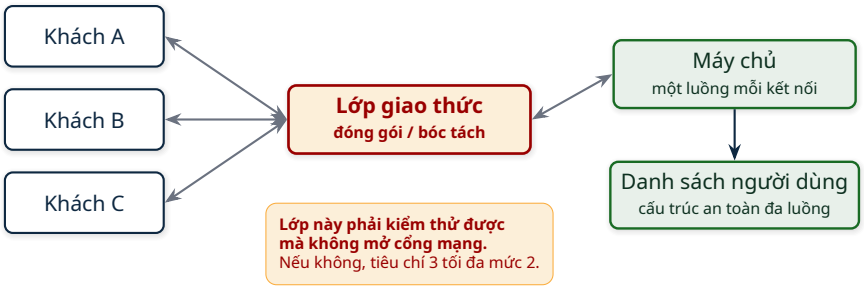


Hình 4.1. Hai bản tin gửi liên tiếp có thể tới bên nhận thành bất kỳ cách chia nào. Cách chữa: tự đặt ranh giới bằng bốn byte độ dài ở đầu mỗi bản tin.

⚠ LƯU Ý

Trên máy cục bộ với tin nhắn ngắn, lỗi này hầu như *không bao giờ* lộ ra. Nó chỉ lộ khi ra mạng thật, khi tin dài, hoặc đúng lúc chấm điểm. Vì vậy khung khoá phần này lại bằng một dòng byte cố ý trả về từng byte một.

4.3 Kiến trúc phải đạt



Hình 4.2. Lớp giao thức nằm giữa và tách rời hẳn khỏi `Socket`. Đó là điều kiện để kiểm thử được giao thức mà không cần mạng.

4.4 Phải làm

PHẢI LÀM

- Máy chủ nhận nhiều kết nối cùng lúc, mỗi kết nối một luồng, phục vụ được ít nhất 20 kết nối đồng thời.
- Lệnh: đặt biệt danh, vào phòng, gửi tin chung, gửi tin riêng, xem ai đang trực tuyến, rời phòng.
- Tin nhắn của một người phải đến mọi người trong cùng phòng.
- Chương trình khách chạy ở dòng lệnh.
- Ghi nhật ký máy chủ ra tệp.
- Danh sách người dùng dùng cấu trúc an toàn đa luồng, *không* bao một khoá `synchronized` quanh toàn bộ máy chủ.

TRƯỜNG HỢP BIÊN

Khách thoát đột ngột (rút mạng, đóng cửa sổ) · hai người trùng biệt danh · tin nhắn rỗng · tin nhắn rất dài · hai tin gửi liên tiếp trong một phần nghìn giây.

4.5 Chấm thêm

Khách thoát đột ngột không được làm sập máy chủ. Đây là chỗ dễ mất điểm: một `IOException` chưa bắt trong luồng phục vụ có thể kéo theo cả tiến trình.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá lớp đóng gói và bóc tách bản tin: gói rồi bóc lại, hai bản tin liên tiếp không dính nhau, bản tin rỗng, bản tin 100000 ký tự, hết dòng trả về `null`, và đọc đúng cả khi dòng byte trả về từng byte một. Sáu phép thử.

Mở rộng tự chọn. Nhiều phòng cùng lúc. Lịch sử tin nhắn khi vào phòng muộn. Chặn người dùng.

5

Đề 03 — Công cụ phân tích nhật ký máy chủ

◎ MỤC TIÊU CHƯƠNG

Độ khó: vừa · **Chương nặng nhất:** 2, 3 · **Khung:** `src/de03_phantichlog/T03KhungPhanTichLog.java`

Đề này đo khả năng dùng Stream đúng chỗ, và đo một thứ ít ai để ý: kết quả báo cáo có *ổn định* giữa hai lần chạy hay không.

5.1 Bối cảnh

Đội vận hành cần báo cáo hằng ngày từ nhật ký máy chủ web: đường dẫn nào bị gọi nhiều nhất, địa chỉ nào gây nhiều lỗi nhất, lưu lượng theo giờ, và thời gian phản hồi của từng đường dẫn.

Mỗi dòng nhật ký có sáu trường, cách nhau bằng khoảng trắng:

```
10.0.0.1  2026-08-20T10:15:00  GET  /trang  200  37
```

5.2 Phần lõi: đường ống xử lý và chỗ nó vỡ



Hình 5.1. Đường ống xử lý nhật ký. Bốn bước đầu ai cũng làm đúng; bước thứ tư là chỗ làm hỏng báo cáo một cách âm thầm.

⚠ LƯU Ý

Yêu cầu *một triệu dòng* trong đề không phải để doạ. Nó có mục đích rất cụ thể: nó làm cho `Files.readAllLines` không dùng được nữa, và buộc bạn phải dùng `Files.lines` cùng `try`-with-resources.

5.3 Phải làm

PHẢI LÀM

- Đọc tệp nhật ký theo đúng định dạng sáu trường ở trên.
- Báo cáo: đường dẫn bị gọi nhiều nhất, địa chỉ gây nhiều lỗi nhất, lưu lượng theo giờ trong ngày, thời gian phản hồi trung bình và trung vị của từng đường dẫn.
- Xuất kết quả ra CSV.
- Dòng sai định dạng bị bỏ qua và *đếm lại*, không được làm dừng chương trình.
- Toàn bộ phần tổng hợp dùng Stream và `Collectors`, không dùng vòng lặp thủ công.
- Chạy được với tệp một triệu dòng mà không nạp hết vào bộ nhớ.

TRƯỜNG HỢP BIÊN

Dòng sai định dạng · dòng rỗng · tệp rỗng · hai đường dẫn bằng lượt · mã trạng thái không phải số · thời gian phản hồi bằng 0.

5.4 Chấm thêm

Báo cáo chạy hai lần phải cho kết quả *giống hệt nhau*, kể cả thứ tự các mục bằng nhau. Khi bảo vệ, bạn sẽ được hỏi: nếu đổi sang `groupingBy` một tham số thì hỏng ở đâu.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá: bóc một dòng đúng định dạng, dòng thiếu trường trả về `null`, dòng có mã không phải số trả về `null`, đếm số dòng hỏng, xếp hạng đường dẫn có phân định khi bằng lượt, và trung bình mili giây của một đường dẫn. Sáu phép thử.

Mở rộng tự chọn. Lọc theo khoảng thời gian. Phát hiện địa chỉ bất thường theo số lỗi trên tổng số lượt. Vẽ biểu đồ lưu lượng theo giờ bằng ký tự.

6

Đề 04 — Hàng đợi tác vụ và nhóm luồng xử lý

◎ MỤC TIÊU CHƯƠNG

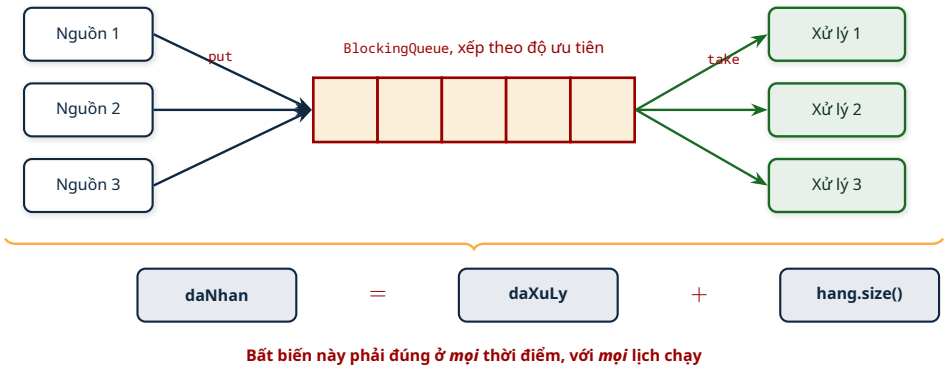
Độ khó: khó · **Chương nặng nhất:** 4 · **Khung:** `src/de04_hangdoitacvu/T04KhungHangDoiTacVu.java`

Phép thử mạnh nhất của đề này không phải là chạy thử thấy chạy được, mà là một *bất biến*: số nhận phải luôn bằng số xử lý cộng số còn lại.

6.1 Bối cảnh

Một hệ nhận tác vụ từ nhiều nguồn (nén ảnh, gửi thư, dựng báo cáo), xếp vào hàng đợi theo độ ưu tiên, và giao cho một nhóm luồng xử lý. Việc đến không đều: có lúc dồn dập, có lúc không có gì.

6.2 Kiến trúc và bất biến phải giữ



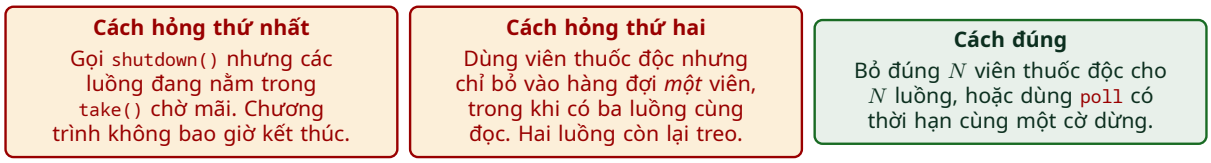
Hình 6.1. Nguồn nộp việc, hàng đợi chặn, nhóm luồng xử lý, và bất biến ràng buộc ba bộ đếm lại với nhau.

GHI NHỚ

Một hệ đa luồng có thể chạy đúng chín lần liền và sai lần thứ mười. Vì vậy thứ duy nhất đáng tin không phải là *đã chạy thử rồi*, mà là một đại lượng không được phép thay đổi dù lịch chạy thế nào.

Nếu số nhận lệch số xử lý cộng số còn lại, thì ở đâu đó bạn đã *làm mất* một tác vụ, hoặc *xử lý nó hai lần*. Cả hai đều là lỗi nặng và cả hai đều không lộ ra khi chạy tay.

6.3 Chỗ bài làm hay treo: dừng hệ



Hình 6.2. Ba cách kết thúc hệ. Hai cách đầu đều làm chương trình treo, và cả hai đều là lỗi phổ biến trong bài nộp.

6.4 Phải làm

PHẢI LÀM

- Nhận tác vụ từ nhiều nguồn đồng thời, xếp theo độ ưu tiên.
- Nhóm luồng xử lý có số lượng đặt được.
- Người nộp và người xử lý chạy trên các luồng khác nhau.
- Dùng `BlockingQueue`, không tự viết vòng lặp chờ bận bằng `sleep`.
- Báo cáo: đã nhận, đã xử lý, đang chờ, tác vụ chậm nhất.
- Lệnh dừng hệ có trật tự: xử lý nốt việc đang chờ rồi mới tắt. Không dùng `Thread.stop` hay ngắt cứng.

TRƯỜNG HỢP BIÊN

Không có việc nào · nhiều luồng hơn việc · một tác vụ ném ngoại lệ · dừng hệ trong khi đang có việc chạy dở.

6.5 Chấm thêm

Bất biến phải được báo cáo qua *năm* lần chạy, không phải một. Với đề đa luồng, phép thử chạy một lần không được tính là bằng chứng.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá vòng lặp của người xử lý và cách dừng hệ, kiểm bằng bất biến trên các quy mô 100, 1000, 0 và 1 tác vụ, cộng một phép thử chạy lặp năm lần. Năm phép thử.

Mở rộng tự chọn. Thử lại tác vụ thất bại với số lần giới hạn. Đo thông lượng theo số luồng xử lý và vẽ bảng so sánh.

7

Đề 05 — Hệ đặt vé xem phim có giữ chỗ

◎ MỤC TIÊU CHƯƠNG

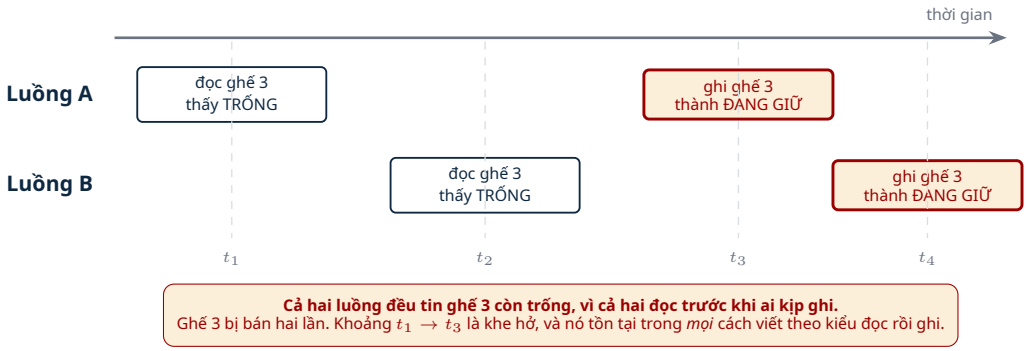
Độ khó: khó · **Chương nặng nhất:** 4 · **Khung:** `src/de05_datve/T05KhungDatVe.java`

Yêu cầu *không bao giờ bán một ghế cho hai người* nghe hiển nhiên. Cái bẫy nằm ở chỗ đọc trạng thái ghế rồi ghi lại là *hai* phép, không phải một.

7.1 Bối cảnh

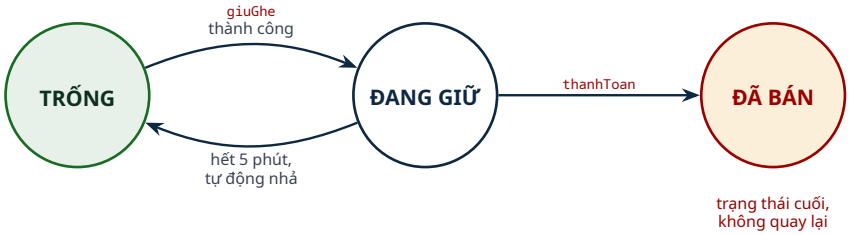
Bán vé trực tuyến cho một rạp chiếu phim. Người dùng xem sơ đồ ghế, chọn ghế, giữ chỗ trong 5 phút, rồi xác nhận thanh toán. Hết 5 phút mà chưa thanh toán thì ghế tự động nhả ra. Nhiều người có thể đặt cùng lúc.

7.2 Phần lõi: đọc rồi ghi không phải một phép nguyên tử



Hình 7.1. Hai luồng cùng giữ một ghế. Khe hở giữa lúc đọc và lúc ghi là toàn bộ vấn đề của đề này.

7.3 Trạng thái một ghế



Hai phép chuyển trạng thái này phải là nguyên tử: `TRỐNG` → `ĐANG GIỮ` và `ĐANG GIỮ` → `ĐÃ BÁN`.
Gợi ý: `AtomicIntegerArray.compareAndSet(ghe, cuMong, moi)` làm cả hai việc đọc và ghi trong một phép duy nhất.
Nếu bạn định dùng `synchronized`, phải trả lời được: khoá trên cái gì, và có bao nhiêu khoá.

Hình 7.2. Ba trạng thái của một ghế và các phép chuyển. Việc nhả ghế hết hạn là tự động, không chờ người dùng bấm nút.

7.4 Phải làm

PHẢI LÀM

- Sơ đồ ghế của một suất chiếu, chọn ghế, giữ chỗ 5 phút, xác nhận thanh toán.
- **Không bao giờ được bán một ghế cho hai người.** Đây là yêu cầu tuyệt đối.
- Giữ chỗ hết hạn phải được nhả tự động.
- Báo cáo: đã bán bao nhiêu vé, đang giữ bao nhiêu, còn trống bao nhiêu.
- Phải kiểm được bằng phép thử nhiều luồng, không chỉ bằng chạy tay.

TRƯỜNG HỢP BIÊN

Hai người bấm cùng một ghế cùng lúc · thanh toán sau khi hết hạn giữ · huỷ giữ · suất hết ghế · thanh toán một ghế chưa ai giữ.

7.5 Chấm thêm

Khi bảo vệ, bạn phải giải thích được đọc rồi ghi trở thành một phép nguyên tử bằng cách nào trong bài của mình. Trả lời *tôi có dùng synchronized* chưa đủ; phải nói rõ khoá đặt trên đối tượng nào và vì sao đủ.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá phép giữ ghế và phép thanh toán, cộng một phép thử cho **200 luồng cùng tranh một ghế**, chạy lặp năm lần, đòi hỏi đúng một luồng thắng ở cả năm lần. Sáu phép thử.

Mở rộng tự chọn. Nhiều suất chiếu. Chọn nhiều ghế liền kề trong một lần. Hoàn vé. Hàng chờ khi hết ghế.

8

Đề 06 — Trình gộp báo giá từ nhiều nguồn

◎ MỤC TIÊU CHƯƠNG

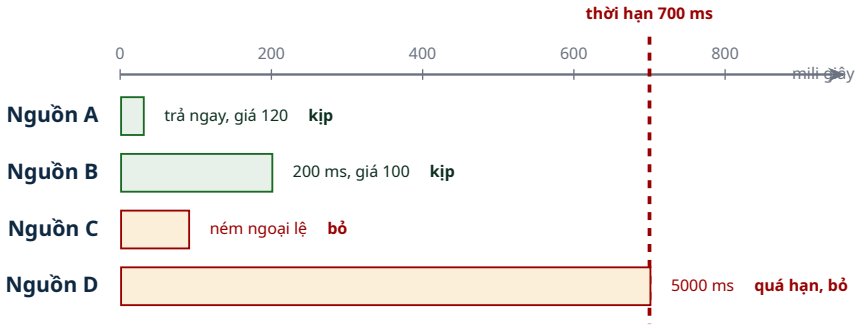
Độ khó: vừa · **Chương nặng nhất:** 4 · **Khung:** `src/de06_gopgia/T06KhungGopGia.java`

Đề này đo việc dùng công cụ chứ không đo vòng lặp. Rất nhiều bài làm chỉ song song *trên giấy tờ*, và khung có một phép thử đo đúng chỗ đó.

8.1 Bối cảnh

So sánh giá một mặt hàng từ nhiều trang bán hàng. Mỗi nguồn là một dịch vụ độc lập, trả lời với tốc độ khác nhau, và thỉnh thoảng hỏng. Công cụ phải trả kết quả trong thời hạn cho trước, lấy những gì kịp về và báo rõ nguồn nào không kịp hoặc lỗi.

8.2 Phần lõi: song song thật và thời hạn thật



Kết quả gộp: `{A=120, B=100}`, giá rẻ nhất `100`. Nguồn C và D không có mặt trong bảng, nhưng phải có mặt trong *báo cáo trạng thái*.

Hình 8.1. Bốn nguồn được hỏi cùng lúc. Thời hạn tính từ lúc gọi hàm gộp, không phải từ lúc từng nguồn bắt đầu chạy.

⚠ LƯU Ý

Cái bẫy lớn nhất của đề này. Nếu bạn dùng `ExecutorService` rồi gọi `get()` lần lượt trên từng `Future` theo thứ tự, tổng thời gian chờ vẫn là *tổng* của các nguồn, chứ không phải nguồn chậm nhất. Khi đó bạn chỉ song song trên giấy tờ.
Khung đo đúng điều này: ba nguồn, mỗi nguồn 400 mili giây, phải xong dưới 1200 mili giây. Xem `invokeAll` có thời hạn, hoặc `CompletableFuture`.

8.3 Phải làm

PHẢI LÀM

- Hỏi nhiều nguồn **song song**, mỗi nguồn là một dịch vụ độc lập.
- Có thời hạn chung. Quá hạn thì bỏ nguồn đó, không chờ vô thời hạn.
- Một nguồn hỏng không được kéo cả kết quả xuống.
- Bảng giá gộp kèm trạng thái từng nguồn: kịp, quá hạn, hay lỗi.
- Ghi lại lịch sử giá và vẽ biểu đồ dạng văn bản.

TRƯỜNG HỢP BIÊN

Mọi nguồn đều hỏng · mọi nguồn đều quá hạn · một nguồn trả về giá âm · chỉ có một nguồn · thời hạn quá ngắn để nguồn nào kịp.

8.4 Chấm thêm

Định nghĩa *quá hạn* phải rõ ràng và đúng: tính từ lúc gọi hàm gộp, không phải từ lúc từng nguồn bắt đầu chạy. Hai cách hiểu này cho kết quả khác nhau khi số nguồn lớn hơn số luồng trong nhóm.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá: bỏ hết nguồn hỏng và nguồn quá hạn, tìm giá rẻ nhất trong số nguồn kịp, thời hạn quá ngắn thì chỉ còn nguồn trả ngay, mọi nguồn hỏng thì bảng rỗng, không nguồn nào kịp thì trả về -1, và **phép đo song song thật**. Sáu phép thử.

Mở rộng tự chọn. Bộ nhớ đệm kết quả theo mã hàng. Thử lại nguồn hỏng một lần. Cảnh báo khi một nguồn lệch giá quá xa trung vị.

9

Đề 07 — Bộ nhớ đệm LRU và đo tỷ lệ trúng

◎ MỤC TIÊU CHƯƠNG

Độ khó: vừa · **Chương nặng nhất:** 3 · **Khung:** `src/de07_bonhodem/T07KhungBoNhoDem.java`

Cả đề nằm ở chỗ ghép *hai* cấu trúc dữ liệu lại với nhau, chứ không ở chỗ tìm ra một lời gọi hàm có sẵn.

9.1 Bối cảnh

Một hàm tính toán đắt, được gọi lại nhiều lần với cùng tham số. Bộ nhớ đệm giữ kết quả gần đây; khi đầy thì loại bỏ phần tử lâu không dùng nhất.

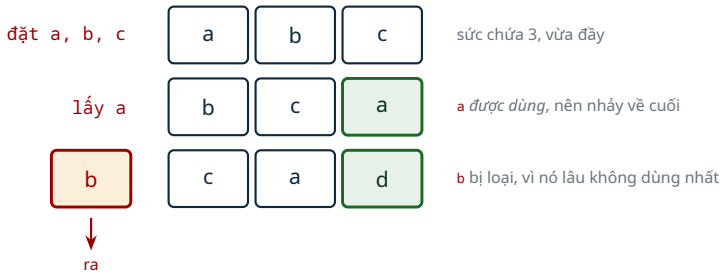
9.2 Phần lõi: một cấu trúc không làm được cả hai việc

Bộ nhớ đệm LRU cần đồng thời hai thứ, và không cấu trúc đơn lẻ nào cho cả hai với chi phí hằng số:



Hình 9.1. Hai yêu cầu mâu thuẫn về cấu trúc, và cách ghép hai cấu trúc để thoả cả hai.

9.3 Thứ tự loại bỏ, chỗ dễ sai nhất



Chỗ sai phổ biến: coi `lay` chỉ là đọc nên không cập nhật thứ tự. Khi đó `a` bị loại thay vì `b`, và mọi con số tỷ lệ trúng đều sai theo mà chương trình vẫn chạy bình thường. Đặt lại một khoá đã có cũng tính là *vừa dùng*.

Hình 9.2. Ba thao tác liên tiếp trên bộ đệm sức chứa 3. Trái là lâu không dùng nhất, phải là mới dùng nhất.

9.4 Phải làm

PHẢI LÀM

- Bộ nhớ đệm sức chứa cố định, loại bỏ phần tử lâu không dùng nhất.
- **Tự cài đặt**, không được gọi thẳng chế độ truy cập của `LinkedHashMap`.
- Đo và báo cáo: số lần trúng, số lần trượt, số lần loại bỏ, tỷ lệ trúng.
- So sánh với một chính sách khác (ví dụ loại ngẫu nhiên, hoặc vào trước ra trước) trên cùng một chuỗi truy cập.
- Chạy được khi nhiều luồng dùng chung một bộ đệm.

TRƯỜNG HỢP BIÊN

Sức chứa 1 · đặt lại một khoá đã có · lấy khoá không có · bộ đệm rỗng · lấy rồi đặt xen kẽ nhau.

9.5 Chấm thêm

Cấm dùng chế độ truy cập của `LinkedHashMap` là *cố ý*. Nếu bài của bạn chỉ gọi một hàm có sẵn thì không có gì để chấm ở tiêu chí 2 và 3.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá: đặt rồi lấy lại, khoá chưa có trả về `null`, thứ tự khi chưa đầy, thứ tự sau khi loại bỏ, khoá vừa lấy trở thành mới dùng nhất, khoá vừa lấy không bị loại, và ba bộ đếm trúng, trượt, loại bỏ. Bảy phép thử.

Mở rộng tự chọn. Chính sách hết hạn theo thời gian. Bộ đệm hai tầng. Đo ảnh hưởng của sức chứa lên tỷ lệ trúng và vẽ bảng.

10

Đề 08 — Mô phỏng chuyển khoản ngân hàng

◎ MỤC TIÊU CHƯƠNG

Độ khó: rất khó, khó nhất trong mười hai đề · **Chương nặng nhất:** 4 · **Khung:** `src/de08_nganhang/T08KhungNganHang.java`

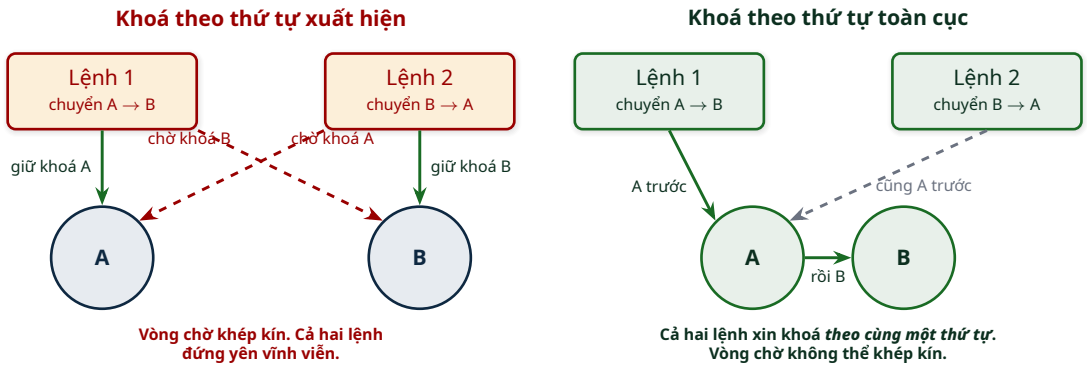
Đây là bài học về khoá chết rõ nhất của cả học phần. Phần lớn thời gian của nhóm sẽ dành cho việc *chứng minh* hệ không thể khoá chết, chứ không phải viết chức năng.

10.1 Bối cảnh

Một ngân hàng có nhiều tài khoản và rất nhiều lệnh chuyển khoản chạy đồng thời. Hệ phải bảo đảm tiền không tự sinh ra và không biến mất, không tài khoản nào âm, và hệ không bao giờ treo.

10.2 Phần lõi: bốn điều kiện sinh khoá chết đều có mặt

Nếu bạn khoá tài khoản nguồn rồi khoá tài khoản đích, thì hai lệnh ngược chiều nhau sẽ ôm nhau chờ mãi mãi.



Hình 10.1. Trái: khoá theo thứ tự nguồn rồi đích sinh ra vòng chờ. Phải: khoá theo một thứ tự toàn cục, ví dụ theo số tài khoản tăng dần, phá vỡ điều kiện vòng chờ.

GHI NHỚ

Bốn điều kiện sinh khoá chết ở chương 4 đều có mặt trong đề này: loại trừ lẫn nhau, giữ và chờ, không tước đoạt, và chờ theo vòng. Bạn chỉ cần phá *một* điều kiện là đủ, và điều kiện dễ phá nhất là điều kiện cuối.

Khi bảo vệ, câu *tôi chạy thử không thấy treo* không được tính. Bạn phải nói được vì sao vòng chờ không thể khép kín trong cách cài đặt của mình.

10.3 Bất biến: tổng tiền không đổi



Hình 10.2. Bất biến tổng tiền. Cùng với điều kiện không tài khoản nào âm, nó là toàn bộ tiêu chuẩn đúng đắn của đề.

10.4 Phải làm

PHẢI LÀM

- Nhiều tài khoản, ít nhất 8 luồng cùng chuyển, ít nhất 100 nghìn lệnh.
- **Không** được dùng một khoá duy nhất cho cả ngân hàng.
- Số dư không bao giờ được âm. Không đủ tiền thì từ chối và không đổi gì cả.
- Sổ kê từng tài khoản và tổng tiền toàn hệ cuối phiên.
- Giải thích được vì sao hệ không thể khoá chết.

TRƯỜNG HỢP BIÊN

Chuyển cho chính mình · chuyển số tiền bằng đúng số dư · chuyển số tiền bằng 0 · hai lệnh ngược chiều giữa cùng hai tài khoản chạy cùng lúc · đọc tổng tiền trong khi đang có luồng chuyển.

10.5 Chấm thêm

Phép đọc tổng tiền cũng phải đúng: đọc trong lúc đang có luồng khác chuyển thì vẫn phải ra đúng tổng, không được bắt gặp một lệnh chuyển ở giữa chừng, tức là lúc tiền đã trừ ở bên này mà chưa cộng bên kia.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá phép chuyển cơ bản, phép từ chối khi không đủ số dư, phép đọc tổng tiền, và một phiên **100 nghìn lệnh trên 8 luồng, lặp 3 lần**, đòi hỏi tổng tiền không đổi, không tài khoản nào âm, và không luồng nào treo. Sáu phép thử.

Mở rộng tự chọn. Lịch sử giao dịch và hoàn tác. Lãi suất theo kỳ. So sánh thông lượng giữa khoá theo tài khoản và một khoá chung, kèm số đo.

11

Đề 09 — Máy chủ chia sẻ tệp

◎ MỤC TIÊU CHƯƠNG

Độ khó: vừa · **Chương nặng nhất:** 5, 6 · **Khung:** `src/de09_maychufile/T09KhungMayChuFile.java`

Đề này có một yêu cầu bảo mật thật, không phải trang trí: chặn đường dẫn leo ngược ra ngoài thư mục gốc.

11.1 Bối cảnh

Chia sẻ tài liệu trong mạng nội bộ. Khách liệt kê tệp, hỏi kích thước, tải về, và tải lên. Giao thức do bạn tự định nghĩa và phải được mô tả bằng văn bản trong báo cáo.

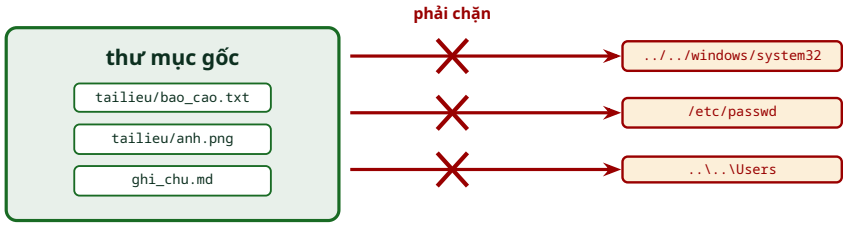
11.2 Giao thức phải tự định nghĩa và tự viết tài liệu



Hình 11.1. Năm lệnh và các mã trả lời. Tài liệu giao thức phải nêu đủ: mỗi lệnh, tham số, mã trả lời, và ví dụ.

11.3 Phần lõi: chặn đường dẫn leo ngược

Một máy chủ tệp không chặn được đường dẫn leo ngược là một máy chủ cho khách đọc *bất kỳ* tệp nào trên ổ đĩa.



Ba dạng phải chặn: leo lên bằng hai dấu chấm, đường dẫn tuyệt đối, và dấu phân cách ngược của Windows.
Cách làm đáng tin: chuẩn hoá đường dẫn về dạng tuyệt đối rút gọn rồi kiểm xem nó có nằm dưới thư mục gốc không. Lọc chuỗi bằng cách tìm hai dấu chấm là cách yếu, vì còn mã hoá URL và liên kết mềm.

Hình 11.2. Đường dẫn khách gửi lên phải được chuẩn hoá và kiểm, không được ghép thẳng vào thư mục gốc.

11.4 Phải làm

PHẢI LÀM

- Lệnh: liệt kê, hỏi kích thước, tải về, tải lên, thoát.
- Tài liệu giao thức bằng văn bản: mỗi lệnh, tham số, mã trả lời, ví dụ.
- Phân biệt được dữ liệu nhị phân với bản tin điều khiển.
- Chặn đường dẫn leo ngược ra ngoài thư mục gốc.
- Nhiều khách tải cùng lúc không làm hỏng nhau.

TRƯỜNG HỢP BIÊN

Tệp không tồn tại · tệp rỗng · tệp lớn hơn bộ nhớ · tên tệp có khoảng trắng · đường dẫn tuyệt đối · lệnh viết thường · lệnh lạ.

11.5 Chấm thêm

Lớp giao thức phải kiểm thử được mà không mở cổng mạng. Nếu để kiểm một lệnh mà phải khởi động máy chủ thật thì tiêu chí 3 tối đa mức 2.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá lớp bóc lệnh và lớp kiểm đường dẫn: lệnh không tham số, lệnh có tham số, lệnh viết thường, dòng rỗng bị từ chối, lệnh lạ bị từ chối, đường dẫn thường là an toàn, đường dẫn leo ngược bị chặn, và định dạng dòng trả lời. Tám phép thử.

Mở rộng tự chọn. Tiếp tục tải dở dang. Băm tệp để kiểm toàn vẹn. Giới hạn tốc độ mỗi khách.

12

Đề 10 — Công cụ chấm bài tập lập trình tự động

◎ MỤC TIÊU CHƯƠNG

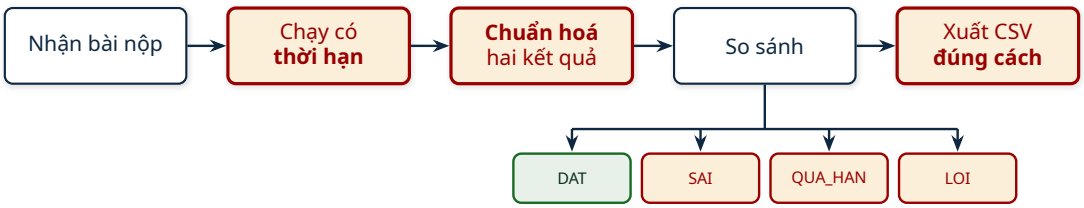
Độ khó: vừa · **Chương nặng nhất:** 4, 6 · **Khung:** `src/de10_chambaitudong/T10KhungChamBaiTuDong.java`

Công cụ chấm bài *là* một phép thử. Và một phép thử hỏng thì nguy hiểm hơn là không có phép thử nào.

12.1 Bối cảnh

Chấm bài nộp của cả lớp. Công cụ nhận một thư mục bài nộp, chạy từng bài với bộ dữ liệu vào cho trước, so kết quả với đáp án, và xuất bảng điểm ra CSV.

12.2 Phần lõi: ba việc dễ làm hỏng công cụ chấm



Ba bước tô đỏ là ba chỗ công cụ hỏng:
Thời hạn · bài chạy mãi không dừng phải bị cắt và tính là trượt, không được làm treo cả công cụ.
Chuẩn hoá · bỏ khoảng trắng cuối dòng, dòng trống cuối tệp, và quy kiểu xuống dòng về một kiểu. Nhưng *giữ nguyên* khoảng trắng ở giữa dòng, vì nó có thể là một phần đáp án.
CSV · tên sinh viên có dấu phẩy phải được bọc trong dấu nháy kép, và dấu nháy kép bên trong phải được nhân đôi.

Hình 12.1. Đường đi của một bài nộp qua công cụ chấm. Ba bước tô đỏ là ba chỗ mà công cụ chấm tự làm hỏng chính nó.

12.3 Chuẩn hoá: khác biệt nào vô hại, khác biệt nào không



Hình 12.2. Ranh giới giữa hai loại khác biệt. Bỏ qua quá tay thì bài sai được điểm; bỏ qua quá ít thì bài đúng bị trượt.

GHI NHỚ

Chính công cụ chấm phải được kiểm: cho nó một bài *cố ý sai* và đòi hỏi nó báo trượt, cho nó một bài đúng và đòi hỏi nó báo đạt. Đây đúng là mục `CHUNG DUONG` mà bạn đã thấy trong mọi khung đề của học phần này.

12.4 Phải làm

PHẢI LÀM

- Nhận thư mục bài nộp, chạy từng bài với bộ dữ liệu vào, so với đáp án.
- Bài chạy quá thời hạn phải bị dừng và tính là trượt, không làm treo công cụ.
- Bỏ qua khác biệt vô hại theo đúng ranh giới ở Hình 12.2.
- Xuất bảng điểm CSV đúng định dạng, kể cả khi tên có dấu phẩy.
- Chấm hai lần trên cùng bài nộp phải ra cùng điểm.

TRƯỜNG HỢP BIÊN

Bài không dịch được · bài chạy mãi không dừng · bài in ra rất nhiều · bài không in gì · bài ném ngoại lệ · đáp án rỗng.

12.5 Chấm thêm

Yêu cầu *chấm hai lần ra cùng điểm* loại bỏ mọi thứ phụ thuộc vào thời gian chạy hay thứ tự duyệt. Nếu điểm đổi giữa hai lần chấm thì công cụ vô dụng, vì không ai bảo vệ được kết quả trước sinh viên khiếu nại.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá bốn hàm: chuẩn hoá, so khớp, chấm có thời hạn, và ô CSV. Chín phép thử, trong đó có bài treo bị cắt theo thời hạn, bài ném ngoại lệ, và ô CSV chứa cả dấu phẩy lẫn dấu nháy kép.

Mở rộng tự chọn. Chấm nhiều bài song song. Phát hiện hai bài nộp giống nhau bất thường. Chấm theo nhiều bộ dữ liệu và cho điểm từng phần.

13

Đề 11 — Công cụ xếp lịch phòng học

◎ MỤC TIÊU CHƯƠNG

Độ khó: vừa · **Chương nặng nhất:** 3 · **Khung:** `src/de11_lapphonghoc/T11KhungLapPhongHoc.java`

Chỗ dễ sai nhất không phải thuật toán xếp, mà là phép kiểm hai khung giờ có chồng lấn nhau hay không.

13.1 Bối cảnh

Phòng đào tạo xếp lịch đầu mỗi học kỳ. Đầu vào là danh sách yêu cầu: môn, giảng viên, sĩ số, các khung giờ có thể dạy. Đầu ra là lịch đã xếp vào phòng cụ thể, hoặc danh sách yêu cầu không xếp được kèm lý do.

13.2 Phần lõi: bốn trường hợp chồng lấn

Rất nhiều bài làm viết phép kiểm chồng lấn thành bốn câu `if` lồng nhau và bỏ sót đúng một trường hợp, thường là trường hợp một khung nằm gọn trong khung kia.



Một dòng thay cho bốn câu if:

```
chongNhau(A, B) = A.batDau < B.ketThuc && B.batDau < A.ketThuc
```

Dùng dấu bé hơn chứ không phải bé hơn hoặc bằng, vì lớp trước vừa xong thì lớp sau vào được ngay.

Hình 13.1. Năm cấu hình hai khung giờ. Bốn câu `if` lồng nhau thường bỏ sót cấu hình thứ ba, và luôn nhầm cấu hình cuối.

13.3 Ba ràng buộc và ba lý do từ chối



Yêu cầu không xếp được phải kèm lý do cụ thể, không được chỉ báo thất bại. Phòng đào tạo cần biết nên mở thêm phòng, đổi giờ, hay tách lớp; ba lý do dẫn tới ba hành động khác nhau.

Hình 13.2. Ba ràng buộc cứng và ba lý do từ chối tương ứng.

13.4 Phải làm

PHẢI LÀM

- Đầu vào: môn, giảng viên, sĩ số, các khung giờ có thể dạy, danh sách phòng.
- Xếp không đụng độ phòng, không đụng độ giảng viên, đủ sức chứa.
- Ưu tiên phòng có sức chứa nhỏ nhất mà vẫn đủ, để dành phòng lớn cho lớp lớn.
- Yêu cầu không xếp được phải kèm lý do cụ thể.
- In lịch theo phòng và theo giảng viên.
- Cùng đầu vào phải cho cùng đầu ra.

TRƯỜNG HỢP BIÊN

Hai khung giờ chạm nhau ở đầu mút · một khung nằm gọn trong khung kia · sĩ số vượt mọi phòng · không còn phòng nào trống · danh sách yêu cầu rỗng.

13.5 Chấm thêm

Yêu cầu *cùng đầu vào cho cùng đầu ra* là để cấm bạn duyệt một `HashSet` rồi xếp theo thứ tự bất kỳ. Nếu kết quả đổi giữa hai lần chạy thì chính bạn cũng không chấm được bài của mình.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá phép kiểm chồng lấn với cả bốn cấu hình, và phép xếp một yêu cầu với ba lý do từ chối cùng quy tắc ưu tiên phòng nhỏ nhất đủ chỗ. Tám phép thử.

Mở rộng tự chọn. Ưu tiên phòng gần nhau cho cùng một giảng viên. Tối thiểu hoá số phòng dùng. Ràng buộc mềm: giảng viên nêu giờ ưa thích.

14

Đề 12 — Cờ ca rô hai người qua mạng

◎ MỤC TIÊU CHƯƠNG

Độ khó: vừa · **Chương nặng nhất:** 1, 5 · **Khung:** `src/de12_cotucao/T12KhungCoCaro.java`

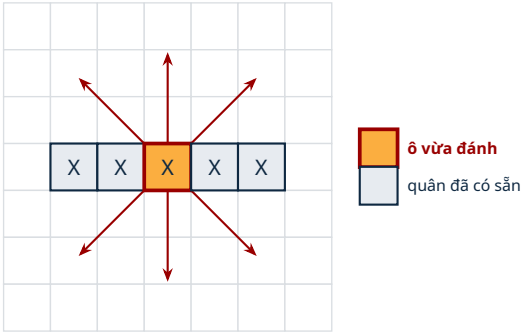
Cái khó không ở phần mạng mà ở hàm kiểm tra thắng. Và yêu cầu tách luật khỏi mạng là một yêu cầu kiến trúc, được chấm điểm.

14.1 Bối cảnh

Hai người chơi trên hai máy khác nhau. Một bên làm máy chủ, bên kia kết nối vào. Hai bên luân phiên đi, bàn cờ đồng bộ, hệ tự phát hiện thắng thua hoà.

14.2 Phần lõi: chỉ xét quanh ô vừa đánh

Duyệt cả bàn cờ sau mỗi nước là cách dễ nghĩ ra nhất và cũng là cách chậm nhất. Cách gọn hơn: chỉ xét bốn hướng đi qua ô vừa đánh, mỗi hướng đếm về hai phía.



Bốn hướng, mỗi hướng đếm về hai phía từ ô vừa đánh: ngang, dọc, chéo xuống, chéo lên.
Số quân liên tiếp = 1 (chính ô đó) + số quân cùng loại về phía trước + số quân cùng loại về phía sau.
Hai chỗ sai kinh điển: đếm nhầm chính ô vừa đánh hai lần, và không dừng lại ở mép bàn cờ.

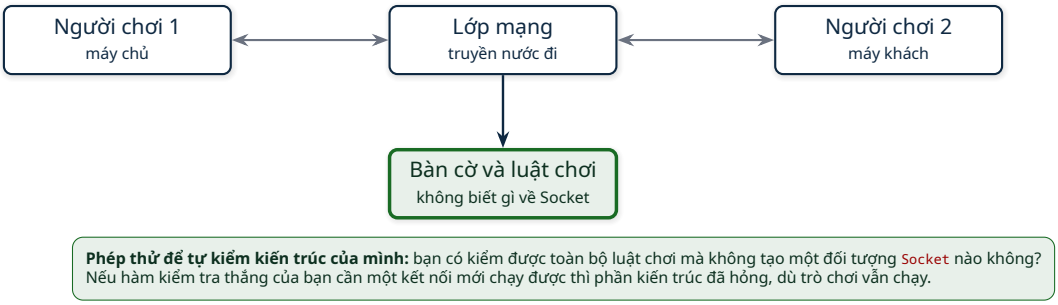
Hình 14.1. Chỉ cần xét bốn hướng đi qua ô vừa đánh. Ở đây năm quân liên tiếp theo hàng ngang, nên nước này thắng.

14.3 Hai trường hợp phải phân biệt được



Hình 14.2. Đếm tổng số quân trên hàng không phải là kiểm tra thắng. Phải là năm quân *liền nhau*.

14.4 Kiến trúc: luật tách hẳn khỏi mạng



Hình 14.3. Lớp luật nằm dưới và độc lập hoàn toàn với lớp mạng.

14.5 Phải làm

PHẢI LÀM

- Một bên làm máy chủ, bên kia kết nối vào.
- Luân phiên đi, bàn cờ đồng bộ hai bên, phát hiện thắng thua hoà.
- Luật thắng: năm quân liền nhau theo hàng, cột, hoặc chéo.
- Nước đi không hợp lệ bị từ chối, và lượt vẫn thuộc về người đó.
- Xem lại ván đấu từng nước một.
- Bàn cờ và luật chơi tách rời hoàn toàn khỏi phần mạng.

TRƯỜNG HỢP BIÊN

Đánh vào ô đã có quân · đánh ra ngoài bàn cờ · quân không phải X hay O · mất kết nối giữa ván · năm quân bị ngắt quãng · bàn cờ đầy mà chưa ai thắng.

14.6 Chấm thêm

Hàm kiểm tra thắng phải chạy được mà không cần đối tượng `Socket` nào. Đây là tiêu chuẩn cụ thể để chấm tiêu chí 3, và cũng là cách bạn tự kiểm kiến trúc của mình trước khi nộp.

PHẦN LÕI, ĐÃ KHOÁ TRONG KHUNG

Khung khoá phép đánh quân và hàm kiểm tra thắng: đánh ô trống, đánh đè, đánh ra ngoài bàn, quân không hợp lệ, năm quân theo hàng, năm quân theo đường chéo, bốn quân chưa thắng, và **năm quân bị ngắt quãng thì chưa thắng**. Tám phép thử.

Mở rộng tự chọn. Máy đánh với người bằng tìm kiếm đối kháng. Nhiều ván cùng lúc trên một máy chủ. Đồng hồ đếm giờ cho mỗi bên.